Autonomous QA Tester Agent

Complete Learning Notebook, Architecture Reference & Technical Interview Guide

A rigorous, code-grounded engineering study guide covering autonomous perception-action loops, deterministic ground-truth verification, false-positive reduction via 2/2 replay gates, and real-world cloud LLM fault tolerance.

Target System:	Node.js Express E-Commerce Store (In-Memory State)
Agent Core:	Python 3.13, Playwright (Chromium), Pydantic v2
LLM Provider:	Google Gemini (gemini-2.5-flash / gemini-3.7-flash via Google GenAI SDK)
Test Suite:	22 Automated Pytest Tests (100% Deterministic Passing)
Documentation Scope:	12 In-Depth Chapters Grounded in Live Repository Codebase

Table of Contents

1. Project Overview & Core Philosophy — The problem solved, autonomous QA definition, workflow, capabilities and limits, architecture diagram.

2. Repository Architecture & File-by-File Breakdown — Comprehensive breakdown of all modules (app.py, agent.py, browser.py, models.py, server.js, tests).

3. Step-by-Step Code Walkthrough & Deep Dive — Classes, methods, inputs/outputs, side effects, Pydantic contracts, Playwright DOM snapshots, error handling.

4. End-to-End Walkthrough of a Real Test Case — Detailed execution trace of Rule R1 (Negative Quantity Rejected) from spec reading to replay confirmation.

5. Technology Stack & Dependency Deep Dive — Python, Google Gemini, GenAI SDK, Playwright, Pydantic, Node/Express, pytest, python-dotenv.

6. Autonomous vs. Deterministic vs. Hardcoded Systems — Detailed 5-column comparative matrix classifying all 10 architectural components.

7. Deep Dive into Real Evaluation Results — Analysis of historical runs, clean baseline runs, latest demonstration run, and metric definitions.

8. Debugging & Troubleshooting Playbook — Diagnosing and fixing Gemini 503/429 errors, API keys, browser failures, server unreachable, replay failures.

9. Practical Setup, CLI Reference & Commands — Windows PowerShell and Bash commands for setup, demo shop startup, agent execution, and pytest.

10. Hands-On Modification Tutorials — Six progressive exercises with step-by-step instructions, code diffs, verification tests, and undo steps.

11. Comprehensive AI Engineering Interview Preparation — Elevator pitches, 25 in-depth technical questions with model answers, and transparent AI tool disclosure.

12. Self-Assessment, Technical Glossary & Mastery Checklist — 20-term glossary, 15-question quiz with answers, code challenge, and 15-point mastery checklist.

1. Project Overview & Core Philosophy

1.1 The Core Problem: Why Conventional Automated QA Breaks

Traditional web test automation (e.g., standard Selenium, Cypress, or Playwright scripts) relies on hardcoded, linear sequences of actions. A developer or QA engineer writes explicit step-by-step instructions: navigate to a URL, find an element with a specific XPath or CSS selector, click it, type a string, and assert that a specific DOM element contains expected text. While effective for repetitive regression testing of static flows, this paradigm suffers from three fundamental flaws:

- **1. Flakiness & Maintenance Burden:** Fragile Element Selectors: Minor UI redesigns, updated CSS class names, dynamic framework-generated IDs, or layout restructuring immediately break conventional scripts, requiring continuous manual maintenance.
- **2. Zero Adaptability:** Lack of Exploratory Capability: A scripted test only checks exactly what its author anticipated. It cannot discover edge cases outside its hardcoded path, cannot adapt if an unexpected modal appears, and cannot navigate alternative routes to a goal.
- **3. Resource Intensity:** High Authoring Cost: Writing comprehensive end-to-end test suites for complex e-commerce or SaaS applications demands hundreds of engineering hours, leading to sparse coverage and missed bugs.

1.2 What Is an Autonomous QA Testing Agent?

An Autonomous QA Testing Agent is an AI system that perceives a software interface, reasons about its requirements against a plain-English specification, decides its own actions dynamically, and independently verifies whether the application behaves correctly. Rather than following a pre-recorded script, the agent behaves like a human QA engineer sitting in front of a web browser:

- • **Spec-Driven:** It reads a specification written in natural language (Markdown) describing business rules and expected behaviors.
- • **Perception-Reasoning Loop:** At every step, it inspects the live browser DOM, extracts interactive elements into a compact snapshot, and asks a Large Language Model (LLM): 'Given this current screen and our testing objective, what action should we take next?'
- • **Dynamic Execution:** It resolves element references to stable CSS locators, executes the action in a real Chromium browser, and observes the outcome.
- • **Rigorous Verification:** When a potential bug is detected, it does not trust LLM intuition alone—it triggers deterministic server-state verification and independent replays to ensure zero false positives.

1.3 End-to-End Workflow: From Demo App to Markdown Report

The agent operates as a structured, 6-stage pipeline from initial setup to report compilation:

1. **Step 1 (Application Startup):** The target Express e-commerce application is started locally on port 3000 (either in clean mode with no bugs, or with specific planted bugs enabled).
2. **Step 2 (Spec Understanding & Ambiguity Detection):** The agent reads specs/shop.md, asks the LLM to extract testable specification rules (R1, R3, R4, R5), and flags any deliberate ambiguities (asking the user for clarification if needed).
3. **Step 3 (Test Planning):** The LLM synthesizes 1 to 6 targeted test cases covering each rule and saves the plan to reports/test_plan.json.

4. **Step 4 (Autonomous Browser Execution):** The agent executes each test case sequentially in a fresh browser context. In each step of a bounded loop (max 10 actions), it builds a DOM snapshot, queries Gemini for the next action, and executes it via Playwright.
5. **Step 5 (Deterministic State Judgment):** Upon completing actions, the agent queries the application's internal state (GET /__state) and runs deterministic Python rule assertions (app_checks()).
6. **Step 6 (Independent Replay Gate & Reporting):** For any suspected violation, the agent resets server state (POST /__reset) and deterministically replays the recorded CSS action sequence twice in fresh browser contexts without the LLM. Only bugs reproducing in 2/2 attempts are confirmed.
7. **Step 7 (Report Generation):** The agent writes reports/bug_report.md containing an executive summary, a complete Test Execution Details table for all attempted tests, reproduction steps, and screenshot evidence.

> **[ARCHITECTURAL PHILOSOPHY]**
> Key Interview Takeaway: The agent uses the LLM for exploration and planning (where probabilistic reasoning excels) but relies strictly on deterministic Python code and server-state assertions for bug judgment and reproduction (where 100% precision is required). This hybrid architecture eliminates LLM hallucinations in test reporting.

1.4 Conventional Scripted Testing vs. Autonomous QA Agent

Dimension	Conventional Automated Script	Our Autonomous QA Agent
Test Authoring	Engineers manually write code for every click and fill.	LLM autonomously generates test cases from plain-English spec.
Navigation	Hardcoded element IDs and fixed sequential paths.	Dynamic action selection based on current DOM snapshot.
UI Resilience	Breaks if layout, class names, or element order shift.	Reads visible interactive elements dynamically; adapts paths.
Ambiguity Handling	Requires human engineer to clarify requirements before coding.	Agent autonomously detects ambiguous rules and asks user.
Bug Judgment	DOM assertion checks written by engineer.	Deterministic server-state queries against mathematical rules.
False Positives	Flaky network or race conditions cause intermittent alerts.	Strict 2/2 independent replay gate filters out transient anomalies.
Reporting	Basic pass/fail stack traces.	Complete Markdown report with root-cause error reasons and screenshots.

1.5 Current Capabilities vs. Deliberate Boundaries

To maintain explainability and suitability for an intern project, the system implements well-defined capabilities and deliberate boundaries:

- **Confirmed Capabilities:** Extracts explicit rules, identifies missing requirements, generates bounded test plans, drives Chromium via Playwright, computes stable locators, enforces action/LLM budgets, and replays bugs without an LLM.
- **Deliberate Boundaries:** Single-origin localhost only (http://localhost:3000); hardcoded app_checks() assertions tailored to demo shop schema; no visual pixel diffing; bound to cloud LLM uptime.

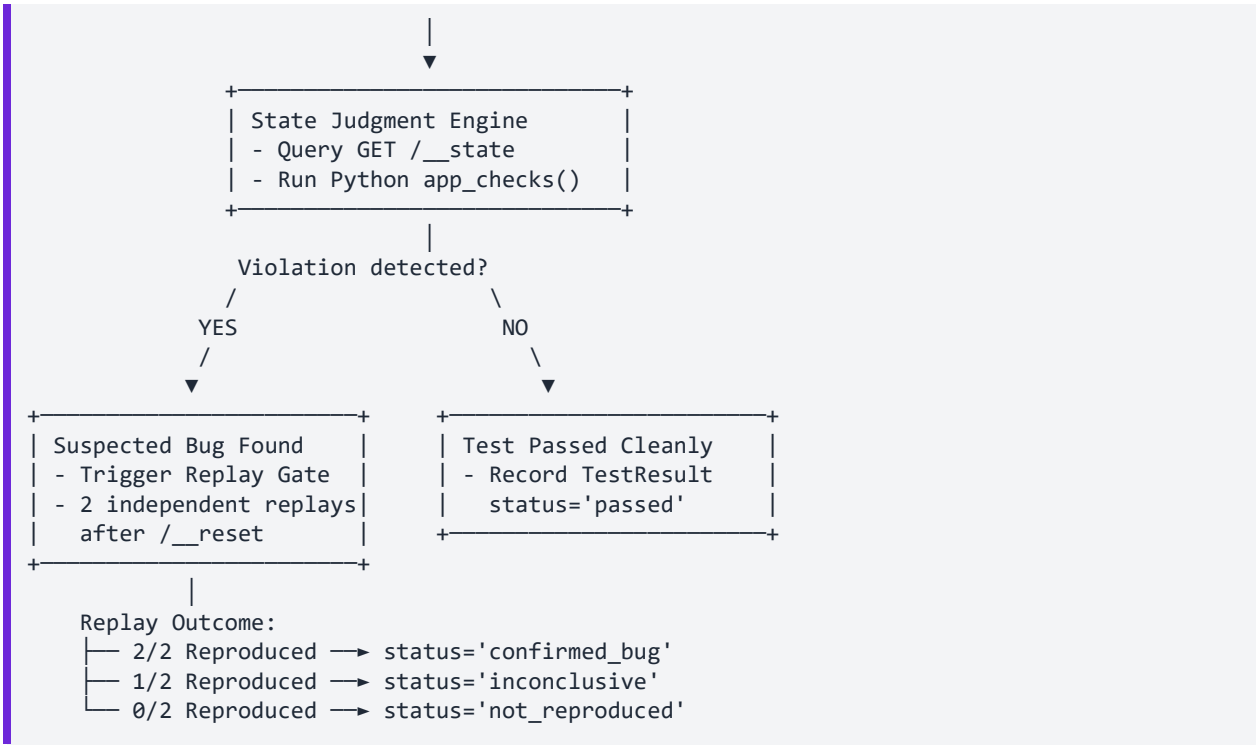
1.6 Architecture & Execution Flow Diagrams

Figure 1.1: High-Level Architectural Pipeline:

```
specs/shop.md  (Plain-English Specification)
       │
       ▼
 [understand_spec()]  ──► Extracts Rules (R1, R3, R4, R5) + Ambiguity Question
       │
       ▼
[generate_test_plan()] ──► 1 to 6 Test Cases (reports/test_plan.json)
       │
       ▼
  [run_test_case()] × N  (Bounded Loop: max 10 actions)
   ├── [take_snapshot()]      ──► Fixed JS DOM injection → compact element list
   ├── [generate_structured()]──► LLM selects AgentAction (Pydantic validated)
   └── [execute_action()]     ──► Playwright executes; records stable locator (#id)
       │
       ▼
 [judge_finding()]    ──► Deterministic app_checks() via GET /__state
       │
       ▼
 [reproduce_bug()] × 2 ──► POST /__reset → fresh context → replay stable locators (No LLM)
       │
       ▼
[generate_markdown_report()] ──► reports/bug_report.md + Screenshot Evidence
```

Figure 1.2: Per-Test Execution State Machine:

```
+-------------------------------------------------------------+
| Start Test Case (e.g. T1 [R1])                              |
|   1. POST /__reset (clean in-memory state)                  |
|   2. browser.new_context() (isolated cookies & cache)       |
+-------------------------------------------------------------+
                              |
                              ▼
               +-------------------------------+
               | Action Loop (max 10 steps)  |<---------------+
               | - Injected JS reads DOM     |                |
               | - LLM picks AgentAction     | (Next action) |
               | - Playwright executes action|                |
               +-------------------------------+---------------+
                              |
                   (action == 'finish' OR max actions reached)
```


State Judgment Engine
- Query GET /__state
- Run Python app_checks()
Violation detected?
YES
NO
Suspected Bug Found
- Trigger Replay Gate
- 2 independent replays after /__reset
Test Passed Cleanly
- Record TestResult
status='passed'
Replay Outcome:
2/2 Reproduced → status='confirmed_bug'
1/2 Reproduced → status='inconclusive'
0/2 Reproduced → status='not_reproduced'

2. Repository Architecture & File-by-File Breakdown

2.1 Repository Directory Map

```
.
├── qa_agent/                    # Python Agent Package
│   ├── __init__.py             # Package marker (enables python -m qa_agent)
│   ├── __main__.py             # Package execution entry point
│   ├── app.py                  # CLI runner & test orchestration pipeline
│   ├── agent.py                # LLM client, spec planning, loop, judging, reporting
│   ├── browser.py              # Playwright controller, DOM snapshots, stable locators
│   └── models.py               # Pydantic data models for every structure
├── demo_shop/                   # Target Node.js Express Application
│   ├── server.js               # Express server, in-memory state, BUGS toggles, /__reset,
/__state
│   ├── package.json            # Node dependencies and start scripts
│   ├── package-lock.json       # Locked npm dependency versions
│   └── public/                 # Static frontend assets
│       ├── index.html          # Product catalog
│       ├── cart.html           # Shopping cart & checkout form
│       ├── signup.html         # User registration page
│       ├── style.css           # Minimal styling
│       └── app.js              # Frontend AJAX logic & DOM rendering
├── specs/                       # Specification Directory
│   └── shop.md                 # Plain-English feature requirements (R1, R3, R4, R5)
├── tests/                       # Unit and Regression Test Suite
│   └── test_agent.py           # 22 pytest unit & regression tests
├── reports/                     # Output Artifacts & Evidence
│   ├── bug_report.md           # Generated bug report
│   ├── demo_all_bugs.md        # Reference evaluation report (latest run)
│   ├── eval_allbugs_run1.md    # Historical run 1 (all bugs planted)
│   ├── eval_allbugs_run2.md    # Historical run 2 (all bugs planted)
│   ├── eval_clean_run1.md      # Historical run 1 (clean app baseline)
│   ├── eval_clean_run2.md      # Historical run 2 (clean app baseline)
│   ├── test_plan.json          # Generated test plan from latest run
│   ├── run_log.jsonl           # Detailed JSONL step execution log
│   └── *.png                   # Captured screenshot evidence (bug_r1.png, etc.)
├── answers.example.json         # Pre-filled answers for non-interactive test runs
├── pytest.ini                   # Pytest configuration
├── requirements.txt             # Python library dependencies
├── README.md                    # Comprehensive project documentation
└── .env.example                 # Template for environment variables (GEMINI_API_KEY)
```

2.2 Detailed File Responsibilities

1. qa_agent/app.py (CLI Orchestrator & Entry Point)

This module is the command-line orchestrator that ties the entire system together. It parses CLI flags (--url, --spec, --answers, --headless, --report, --max-calls), verifies that the demo application is reachable via _wait_for_server(), executes the planning stage, drives the test execution loop, invokes independent bug reproduction, and triggers markdown report generation.

2. qa_agent/agent.py (Intelligence & Decision Engine)

The core logic engine containing: LLMClient (managing Gemini API calls, schema validation, call budgeting, and one corrective retry), understand_spec() (extracting rules and ambiguity questions), resolve_clarification() (handling user answers), generate_test_plan() (synthesizing test cases), run_test_case() (the 10-action agent loop with loop detection), app_checks() and judge_finding() (deterministic server-state evaluation), reproduce_bug() (independent replay gate), and generate_markdown_report() (universal reporting pipeline).

3. qa_agent/browser.py (Browser Driver & DOM Engine)

Manages Playwright Chromium browser instances. Contains BrowserController with methods: start(), close(), new_context(), take_snapshot() (injects _BUILD_SNAPSHOT_JS into the page to extract interactive elements and text), execute_action() (executes clicks, fills, selects, presses, gotos, while resolving and recording stable CSS locators), replay_actions() (executes recorded locator sequences without an LLM), and take_screenshot().

4. qa_agent/models.py (Data Models & Schema Enforcement)

Defines strict Pydantic v2 schemas for all system inputs and outputs: SnapshotElement, PageSnapshot, AgentAction, RecordedAction, BrowserEvidence, SpecRule, SpecUnderstanding, TestCase, TestPlan, BugFinding, and TestResult. All LLM responses are validated against these models, guaranteeing structured execution.

5. demo_shop/server.js (The Target Web Application)

A lightweight Node.js Express server that simulates an e-commerce shop. It maintains in-memory state (cart, appliedCoupons, registeredEmails, total). It exposes product, cart, coupon, checkout, and registration REST APIs, as well as test control endpoints: POST /__reset (clears state in <5ms) and GET /__state (exposes internal memory for ground-truth inspection). It supports four planted bugs controlled via the BUGS environment variable: coupon_twice, negative_qty, stale_total, bad_email.

6. specs/shop.md (The Feature Specification)

A plain-English Markdown document describing four core business requirements: R1 (positive integer quantities), R3 (SAVE10 coupon applies exactly once), R4 (cart total equals item sum minus coupon discount), and R5 (signup requires valid email format). It deliberately omits the maximum quantity allowed per item to test the agent's ambiguity detection.

7. tests/test_agent.py (Automated Test Suite)

A suite of 22 unit and regression tests written in pytest. It verifies deterministic logic without requiring live LLM calls: Pydantic action validation, external URL blocking, call budget capping, JSON fence stripping, corrective retry logic, non-interactive answers lookup, rule validation, action budget enforcement, loop detection, cart total math checks, inconclusive replay handling, report generation structure, Node app reset integration, stable locator replays, and all 8 universal report requirements.

3. Step-by-Step Code Walkthrough & Deep Dive

3.1 Module: qa_agent/models.py

All data passed between components is strongly typed using Pydantic BaseModel classes. Pydantic provides automatic runtime validation, parsing, and type coercion. If the LLM generates an invalid action name or malformed JSON, Pydantic raises a ValidationError that triggers corrective retry.

Code Excerpt 3.1: Core Model: AgentAction (LLM Output Contract):

```
AllowedActionType = Literal[
    "click", "fill", "select", "press", "goto", "finish", "ask_user"
]

class AgentAction(BaseModel):
    thought: str = Field(description="Short explanation of why this action was chosen")
    action: AllowedActionType = Field(description="Action name")
    target: Optional[Union[int, str]] = Field(default=None, description="Element number [N]
or locator")
    value: Optional[str] = Field(default=None, description="Text value for fill or select")
    key: Optional[str] = Field(default=None, description="Key name for press, e.g. Enter")
    url: Optional[str] = Field(default=None, description="Local path or URL for goto")
    question: Optional[str] = Field(default=None, description="Clarification question for
ask_user")
```

- • **Why Literal?** Literal['click', ...]: Restricts action names strictly to valid operations. Any hallucinated action like 'execute_script' or 'submit' immediately fails validation.
- • **Target Flexibility:** Union[int, str]: Allows target to be an index number [3] from the snapshot or an explicit CSS locator '#coupon-input'.

Code Excerpt 3.2: Core Model: TestResult (Unified Result Tracking):

```
class TestResult(BaseModel):
    __test__ = False  # Prevents pytest from collecting this class as a test suite
    test_id: str
    rule_id: str
    objective: str
    status: str = "passed"              # passed, failed, confirmed_bug, inconclusive,
unable_to_complete
    execution_status: str = "completed" # completed vs unable_to_complete
    finding_status: Optional[str] = None
    expected: Optional[str] = None
    observed: Optional[str] = None
    error_type: Optional[str] = None    # e.g. Gemini 503 UNAVAILABLE, Timeout
    error_message: Optional[str] = None # Raw error string
    execution_stage: Optional[str] = None
    reproduction_status: Optional[str] = None  # e.g. 2/2, 1/2, 0/2
    screenshot_path: Optional[str] = None
```

Notice the crucial architectural distinction: execution_status tracks whether the test ran to completion, while status and finding_status track whether a specification violation was detected and confirmed. This decouples infrastructure failures (e.g. API 503) from functional test results.

3.2 Module: qa_agent/browser.py

The BrowserController class encapsulates Playwright Chromium operations. A key technical innovation is the DOM snapshot mechanism that compresses full HTML into a lightweight text representation.

Code Excerpt 3.3: DOM Snapshot Injected JavaScript (_BUILD_SNAPSHOT_JS):

```
const INTERACTIVE_SELECTORS = [
    'button', 'a[href]', 'input', 'select', 'textarea',
    '[role="button"]', '[role="link"]', '[tabindex]:not([tabindex="-1"])',
].join(', ');

// Extracts visible interactive elements, assigns 1-based index,
// and computes a deterministic stable locator for each element:
function getStableLocator(el) {
    if (el.id) return '#' + el.id;
    if (el.dataset.testid) return `[data-testid="${el.dataset.testid}"]`;
    if (el.name) return `${el.tagName.toLowerCase()}[name="${el.name}"]`;
    // Text fallback for buttons and links
    return `${el.tagName.toLowerCase()}:has-text("${el.innerText.trim()}")`;
}
```

Why this matters for LLMs: Raw web pages often contain 50,000+ characters of HTML boilerplate (scripts, SVGs, nested divs). Sending raw HTML consumes massive token budgets and confuses LLM reasoning. By extracting only visible interactive elements, the snapshot compresses page state into ~500 tokens of structured text: '[1] button "Add to cart"'.

3.3 Module: qa_agent/agent.py

Key Classes and Functions in agent.py:

- **1. LLMClient:** Wraps the Google Gemini API client. It sets system instructions demanding JSON only, injects the Pydantic schema, tracks global call counts against max_calls (budget 60), and provides corrective retry: if JSON parsing fails once, it feeds the raw error back to Gemini to get corrected JSON.
- **2. understand_spec():** Prompts Gemini with the raw Markdown specification to return a SpecUnderstanding object containing explicit rules and an ambiguity question.
- **3. resolve_clarification():** Checks if an answers file was provided via --answers. If found, reads the answer; otherwise prompts the user interactively on standard input.
- **4. generate_test_plan():** Prompts Gemini to generate 1 to 6 TestCase objects covering each extracted rule, saved to reports/test_plan.json.
- **5. run_test_case():** Executes up to 10 browser actions. Detects loops (if the exact same action and page state repeat 3 times consecutively, it breaks early). Resolves target numbers to stable CSS locators.
- **6. app_checks() & judge_finding():** Queries GET /__state and asserts ground truth rules mathematically in Python (checking quantities, coupon math, email regex).
- **7. reproduce_bug():** Resets application state via POST /__reset, creates a fresh browser context, and deterministically executes the recorded RecordedAction sequence twice. Confirms the bug only on 2/2 reproduction.
- **8. generate_markdown_report():** Takes the list of TestResult records and renders reports/bug_report.md with tables, unverified rule notices, and evidence.

Code Excerpt 3.4: Corrective Retry Pattern in LLMClient:

```
def generate_structured(self, prompt: str, response_model: Type[T]) -> T:
    raw_text = self._call_llm(prompt)
    cleaned = clean_json_text(raw_text)
    try:
        return response_model.model_validate_json(cleaned)
    except ValidationError as err:
        # One corrective retry: feed error message back to the LLM
        retry_prompt = (
            f"{prompt}\n\n"
            f"[CORRECTION REQUIRED]: Your previous output failed schema
validation:\n{err}\n"
            f"Fix the JSON syntax and return ONLY valid JSON matching the requested
schema."
        )
        retry_text = self._call_llm(retry_prompt)
        return response_model.model_validate_json(clean_json_text(retry_text))
```

4. End-to-End Walkthrough of a Real Test Case

To make the agent's execution concrete, this section traces the exact lifecycle of test case T1 targeting Specification Rule R1 (Invalid or negative quantity accepted in cart). This trace is drawn directly from the real demonstration run recorded in reports/demo_all_bugs.md.

4.1 Step 1: Specification Ingestion & Extraction

The agent reads specs/shop.md which states:

```
### Rule 1: Item Quantities
Quantities must be positive integers (greater than or equal to 1).
Setting a quantity to zero or negative must be rejected.
```

understand_spec() calls Gemini, which extracts the rule into a SpecRule object: id='R1', description='Quantity must be a positive integer', expected_behavior='Reject quantities <= 0'.

4.2 Step 2: Ambiguity Resolution

The specification does not specify a maximum allowed quantity. The LLM notices this omission and sets ambiguity_question='What is the maximum quantity allowed per item in the cart?'. resolve_clarification() reads answers.example.json, obtaining answer='10'.

4.3 Step 3: Test Plan Generation

generate_test_plan() asks Gemini for a test case. Gemini returns TestCase T1: id='T1', rule_id='R1', objective='Verify that setting item quantity to zero or negative is rejected', initial_conditions='Clean cart on home page', expected_outcome='Negative quantity rejected', goal='Add an item, go to cart, update quantity to -1, check if rejected'.

4.4 Step 4: Autonomous Browser Navigation Loop

The agent resets state via POST /__reset, creates a fresh browser context, and navigates to http://localhost:3000/. It then begins the action loop:

- **Action 1:** take_snapshot() runs _BUILD_SNAPSHOT_JS. The LLM receives snapshot with '[1] button "Add to cart"' on Notebook. LLM returns AgentAction: thought='Add notebook to cart to test quantity', action='click', target=1. Playwright clicks #add-p1. RecordedAction recorded with stable locator '#add-p1'.
- **Action 2:** Page updates. Snapshot shows '[2] a "Cart (1)"'. LLM returns AgentAction: thought='Navigate to cart page', action='click', target=2. Playwright clicks #nav-cart. RecordedAction recorded with '#nav-cart'.
- **Action 3:** Cart page loads. Snapshot shows '[3] input[number] value="1"' with stable locator '#qty-p1'. LLM returns AgentAction: thought='Attempt to set negative quantity', action='fill', target=3, value='-1'. Playwright fills #qty-p1 with '-1'. RecordedAction recorded with target_locator='#qty-p1', value='-1'.
- **Action 4:** LLM returns AgentAction: thought='Quantity update submitted, finishing test', action='finish'. Action loop terminates.

4.5 Step 5: Deterministic Ground-Truth Judgment

The LLM is NOT asked whether a bug exists. Instead, judge_finding() invokes app_checks('http://localhost:3000'): Python makes an HTTP request to GET /__state. The Express server returns:

```
{
  "cart": [{"id": "p1", "name": "Notebook", "price": 200, "qty": -1}],
  "appliedCoupons": [],
  "registeredEmails": [],
  "total": -200
}
```

In agent.py, app_checks() checks item['qty'] < 1. It discovers item Notebook with qty=-1. Because this violates Rule R1, it constructs a BugFinding: title='Invalid or negative quantity accepted in cart', expected='Positive integer (>= 1)', actual='Cart accepted item Notebook with quantity -1', status='inconclusive'.

4.6 Step 6: Deterministic 2/2 Independent Replay Gate

Before reporting a bug, reproduce_bug() verifies whether the finding is deterministically reproducible: It DOES NOT call the LLM. Instead, it resets the server via POST /__reset and creates a fresh browser context.

8. **Replay Attempt 1:** Replay 1: replay_actions() executes goto('/'), click('#add-p1'), click('#nav-cart'), fill('#qty-p1', '-1'). It calls GET /__state. Result: Violation confirmed (qty=-1).
9. **Replay Attempt 2:** Replay 2: POST /__reset clears state again. replay_actions() repeats the exact sequence in another fresh browser context. It calls GET /__state. Result: Violation confirmed (qty=-1).

Because the bug reproduced in 2 out of 2 attempts, finding.replays is set to '2/2' and finding.status is updated to 'confirmed'. A screenshot is captured to reports/bug_r1.png.

4.7 Step 7: Universal Report Entry & Error Handling

The TestResult record is updated to status='confirmed_bug', reproduction_status='2/2', and passed to generate_markdown_report(). In reports/demo_all_bugs.md, it is rendered into:

- **Executive Summary:** Count incremented: Confirmed Bugs (2/2 Independent Replays): 2.
- **Test Execution Details Table:** | T1 | R1 | Verify setting quantity to negative is rejected | Confirmed Bug | Confirmed violation (2/2 replays): Cart accepted item 'Notebook' with quantity -1 |
- **Confirmed Bugs Section:** Full severity (High), specification reference, steps to reproduce with CSS selectors, and screenshot link.

> **[FAULT TOLERANCE IN ACTION]**
> How Exceptions are Handled: What if Gemini failed during Action 2 with '503 UNAVAILABLE'? The try...except block in app.py catches the exception, classifies it as 'Gemini 503 UNAVAILABLE', creates a TestResult with status='unable_to_complete', and logs it in the report. The remaining test cases (T2..T5) continue executing safely. One failed test never aborts the run.

5. Technology Stack & Dependency Deep Dive

The project is engineered with a clean, decoupled technology stack designed for transparency, fast execution, and zero hidden framework abstractions. This section explains the role of each component, why it was chosen, and what alternatives exist.

Technology / Tool	Exact Version / Role	Why Chosen in This Project	Alternatives Considered & Tradeoffs
Python	3.10+ (Tested on 3.13.5) Core Agent Language	Rich AI/LLM ecosystem, concise syntax, first-class Playwright support, and native Pydantic data validation.	Node.js / TypeScript: Would unify frontend and backend in one language, but Python offers cleaner AI evaluation tooling.
Google Gemini API	gemini-2.5-flash / gemini-3.7-flash LLM Provider	Fast latency (<1.5s per action), strong JSON adherence, cost-effective pricing, and high reasoning capability.	Claude 3.5 Sonnet: Excellent reasoning but higher cost per token; OpenAI GPT-4o: Comparable, but Gemini SDK offers clean native typing.
Google GenAI SDK	google-genai >= 1.0.0 API Client	Official modern SDK from Google; supports strict typed responses, system instructions, and temperature tuning.	LangChain / LlamaIndex: Deliberately avoided. Adding framework overhead obscures agent loop internals and makes debugging harder.
Playwright (Sync)	playwright >= 1.49.0 Browser Automation	Reliable Chromium automation, auto-waiting for DOM events, headless/headed toggles, isolated browser contexts.	Selenium: Legacy, slow, requires separate ChromeDriver binaries; Cypress: Runs inside browser, difficult to orchestrate from Python.
Pydantic v2	pydantic >= 2.0.0 Schema Validation	Guarantees that LLM JSON outputs adhere strictly to typed schemas; provides instant validation errors for corrective retries.	Raw Python dicts / jsonschema: Lacks type hint integration, IDE autocompletion, and robust error message generation.
Node.js & Express	Node 18+ (Node 24 / Express 4) Target Demo Store	Ultra-lightweight web server; maintains state in server RAM, allowing POST /__reset to clear all state in <5ms.	Django / Rails / MongoDB: External databases require complex teardown scripts or migrations, slowing down replays to 5-10 seconds.
pytest	pytest >= 8.3.4 Test Runner	Standard Python test runner with rich assertion rewriting, fixtures (tmp_path,	unittest: Verbose boilerplate; pytest provides concise

		monkeypatch), and clean CLI output.	functional test definitions.
python-dotenv	python-dotenv >= 1.0.0 Environment Config	Safely loads GEMINI_API_KEY from local .env into os.environ without checking secrets into version control.	Hardcoded constants: High security risk; system environment variables: Cumbersome to manage across multiple dev machines.

[INTERVIEW DISTINCTION]
Why No LangChain or AutoGen? In an AI Engineering Intern interview, interviewers want to see that you understand the mechanics of perception, prompt engineering, context management, loop control, and error handling. Using LangChain or AutoGen hides those mechanics behind black-box abstractions. Building the agent from raw Python demonstrates fundamental mastery of agent architecture.

6. Autonomous vs. Deterministic vs. Hardcoded Systems

A critical evaluation criterion for AI engineering submissions is intellectual honesty regarding autonomy. A system is not '100% autonomous'—production agent architectures are hybrid systems that combine autonomous reasoning with deterministic safety rails and fixed configurations. The table below classifies every component in this project.

Component / Behavior	Classification	Code Evidence	Design Rationale	Current Limitation
Spec Understanding & Ambiguity Detection	Autonomous (LLM-Driven)	understand_spec() in agent.py prompts Gemini with raw Markdown and parses SpecUnderstanding.	Enables the agent to ingest arbitrary business requirements without manual test script translation.	Dependent on LLM prompt formatting; extreme spec ambiguities could be overlooked.
Test Planning & Goal Setting	Autonomous (LLM-Driven)	generate_test_plan() in agent.py prompts Gemini to generate 1 to 6 TestCase objects.	Allows the agent to target both happy paths and boundary conditions autonomously.	Limited to 6 test cases per run to prevent runaway API costs.
Next-Action Browser Selection	Autonomous (LLM-Driven)	In run_test_case(), the LLM chooses AgentAction (click, fill, etc.) from live DOM snapshots.	Simulates a human tester exploring the UI dynamically based on visual state.	LLM reasoning can wander if visual cues are unclear or element descriptions are sparse.
Action Schema Validation	Deterministic (Pydantic)	AgentAction.model_validate_json() validates action types, targets, values, and keys.	Guarantees that malformed or unauthorized LLM actions are rejected before execution.	Actions outside AllowedActionType (e.g. drag-and-drop) are not supported.
DOM Snapshotting & Locator Resolution	Deterministic (JavaScript / Python)	_BUILD_SNAPSHOT_JS and resolve_target() compute priority locators (#id > data-testid > text).	Translates ephemeral snapshot numbers into stable locators resilient to DOM order changes.	Requires elements to have accessible IDs, testids, names, or clear inner text.
Browser Automation & Replay	Deterministic (Playwright)	execute_action() and replay_actions() dispatch native Chromium CDP calls.	Eliminates LLM latency and cost during reproduction; guarantees exact execution repeatability.	Bound to single-origin localhost (http://localhost:3000) for safety.

Bug Judgment (app_checks)	Deterministic (Python Code)	app_checks() in agent.py queries GET /__state and asserts mathematical/regex rules.	Eliminates hallucination in bug reporting; server memory represents absolute ground truth.	Hardcoded to demo shop schema; testing another app requires writing a new check adapter.
2/2 Independent Replay Gate	Deterministic (Fixed Logic)	reproduce_bug() in agent.py requires 2/2 successful replays after POST /__reset.	Filters out transient browser glitches, race conditions, and flaky false positives.	A genuinely intermittent bug that reproduces 1/2 will be labeled 'inconclusive'.
Action & LLM Budgets	Configured / Hardcoded	MAX_ACTIONS_PER_TEST = 10, DEFAULT_MAX_CALLS = 60 in agent.py.	Prevents infinite execution loops, API quota exhaustion, and runaway cloud billing.	Complex checkout flows requiring >10 steps will hit the action budget cap.
Loop Detector	Deterministic (Python Code)	run_test_case() tracks (action, state) hashes; triggers break after 3 identical states.	Safeguard against repetitive clicking or stuck forms.	Breaks the test early, resulting in an inconclusive or passed outcome.

7. Deep Dive into Real Evaluation Results

Every evaluation claim in this project is grounded in actual run artifacts stored in the reports/ directory. This section analyzes those results honestly, explaining both successes and real-world failure modes.

7.1 Summary of Historical and Demonstration Runs

Run Name & Config	Tests Attempted	Confirmed Bugs	Incomplete Tests	Observed False Positives	LLM Calls	Reference Report
Historical Run 1 BUGS=all (All 4 planted)	4	4 / 4 (100%) R1, R3, R4, R5	0	0	25 / 60	reports/eval_allbugs_run1.md
Historical Run 2 BUGS=all (All 4 planted)	4	3 / 4 (75%) R1, R3, R5	1 (Gemini 503 spike on R4)	0	23 / 60	reports/eval_allbugs_run2.md
Latest Demonstration Run BUGS=all (All 4 planted)	5	2 / 5 (40%) R1, R5 confirmed (2/2)	3 (Gemini 503 UNAVAILABLE)	0	15 / 60	reports/demo_all_bugs.md
Clean App Run 1 BUGS=none (Baseline)	4	0 (All 4 tests passed)	0	0 (0.0% FP rate)	28 / 60	reports/eval_clean_run1.md
Clean App Run 2 BUGS=none (Baseline)	4	0 (T1 passed cleanly)	3 (429 Daily Quota Hit)	0 (0.0% FP rate)	20 / 60	reports/eval_clean_run2.md

7.2 Deep Interpretation of Metrics

- **1. Tests Passed:** A test that completed its action loop and deterministic assertions with zero violations detected. This confirms compliance with the spec rule.
- **2. Confirmed Bugs (2/2):** A suspected violation that reproduced in 2 of 2 deterministic replays after state reset without an LLM. These represent genuine, independently verified software defects.
- **3. Inconclusive Findings:** A suspected bug that could not achieve 2/2 replays (e.g. 1/2 or replay exception). It is reported transparently as inconclusive to protect against false alarms.
- **4. Tests Unable to Complete:** A test that halted prematurely because of an external API 503/429 error, timeout, or browser crash. CRITICAL: These are NEVER counted as passes or failures. They simply indicate that test execution stopped before conclusive evidence could be gathered.
- **5. False Positive Rate:** Across all completed clean-app runs (eval_clean_run1.md), zero false bugs were confirmed. The 2/2 replay gate successfully prevented false alarms.

> **[INTELLECTUAL HONESTY]**
> Interview Note on Cloud Availability: Do not claim 'The agent achieves 100% test completion under all conditions.' Acknowledge the reality: 'In our latest demonstration run (reports/demo_all_bugs.md), 3 tests were unable to complete because the Gemini API returned 503 UNAVAILABLE during peak demand. Our reporting pipeline honestly flags them as Unable to Complete and lists the unverified rules under Rules Not Fully Verified.' Interviewers respect this engineering maturity.

8. Debugging & Troubleshooting Playbook

When running an autonomous agent, failures can originate from the web application, browser driver, LLM API, or network. This playbook provides step-by-step diagnostic workflows for the most common issues.

8.1 Issue 1: Gemini API 503 UNAVAILABLE (Model Overloaded)

- **Symptoms:** Agent logs: 'FAIL Test could not complete: 503 UNAVAILABLE: The model is overloaded. Please try again later.'
- **Root Cause:** Upstream Google Cloud infrastructure experiencing peak traffic spikes.

10. **Step 1:** Wait 30-60 seconds for traffic to subside.
11. **Step 2:** Switch to an alternative Gemini flash model in .env: set LLM_MODEL=gemini-2.5-flash-lite or gemini-3.7-flash.
12. **Step 3:** Rerun the agent: python -m qa_agent --answers answers.example.json.

8.2 Issue 2: Gemini API 429 RESOURCE_EXHAUSTED (Quota Limits)

- **Symptoms:** Agent logs: '429 RESOURCE_EXHAUSTED: Quota exceeded for quota metric GenerateContent...'
- **Root Cause:** Free-tier API key exceeded rate limits (15 requests/minute) or daily token limits.

13. **Step 1:** Check Google AI Studio dashboard for rate limit status.
14. **Step 2:** Add rate-limiting delays between test cases if running bulk tests.
15. **Step 3:** Ensure MAX_LLM_CALLS is constrained (e.g. --max-calls 30).

8.3 Issue 3: Missing or Invalid GEMINI_API_KEY

- **Symptoms:** RuntimeError: 'GEMINI_API_KEY is not set. Please add it to your .env file or environment variables.'
- **Root Cause:** .env file missing, incorrectly named, or empty API key string.

16. **Step 1:** Verify .env exists in project root: Test-Path .env in PowerShell.
17. **Step 2:** Ensure format: GEMINI_API_KEY=AIzaSy... (no spaces around '=').
18. **Step 3:** Verify loading in Python: python -c "from dotenv import load_dotenv; import os; load_dotenv(); print(bool(os.environ.get('GEMINI_API_KEY')))".

8.4 Issue 4: Target Demo Application Unreachable

- **Symptoms:** CLI output: '[ERROR] Cannot reach http://localhost:3000. Please start the demo shop first...'
- **Root Cause:** Node server not started or crashed on port 3000.

19. **Step 1:** Open a separate terminal window and navigate to demo_shop/.
20. **Step 2:** Start server: npm run start:clean (or npm run start:all-bugs).
21. **Step 3:** Verify connectivity: curl http://localhost:3000/api/products in PowerShell.

8.5 Issue 5: Playwright Browser Startup Failure

- **Symptoms:** playwright._impl._errors.Error: 'Executable doesn't exist at...'
- **Root Cause:** Chromium browser binaries not downloaded after pip install.

22. **Step 1:** Run Playwright install command: playwright install chromium.
23. **Step 2:** Verify browser launch: python -c "from playwright.sync_api import sync_playwright; p=sync_playwright().start(); b=p.chromium.launch(); b.close(); p.stop(); print('OK')".

8.6 Issue 6: Suspected Bug Fails Independent Replay (0/2 or 1/2)

- **Symptoms:** Report classifies finding as 'not_reproduced' (0/2) or 'inconclusive' (1/2).
- **Root Cause:** Violation depended on timing/race condition, or server state was not properly reset.

24. **Step 1:** Inspect reports/run_log.jsonl to examine exact action sequence.
25. **Step 2:** Verify POST /__reset endpoint cleans state by checking GET /__state.
26. **Step 3:** Run agent in visible mode (--no-headless) to observe replay execution visually.

9. Practical Setup, CLI Reference & Commands

This section provides verified, copy-pasteable commands for setting up and operating the system on Windows (PowerShell) and Linux/macOS. Always pay close attention to the working directory (project root vs. demo_shop/).

9.1 Initial Environment Setup (One-Time)

From the project root (x:\QA Testing Agent):

```
# 1. Create a Python virtual environment (if not already present)
python -m venv .venv

# 2. Activate virtual environment
.venv\Scripts\Activate.ps1       # Windows PowerShell
# source .venv/bin/activate       # Linux/macOS

# 3. Install Python dependencies
pip install -r requirements.txt

# 4. Install Playwright browser binaries
playwright install chromium

# 5. Set up environment file
cp .env.example .env
# Edit .env and enter your Gemini API key:
# GEMINI_API_KEY=your_actual_key_here
```

9.2 Setting Up & Starting the Demo Shop

In a separate terminal window:

```
cd demo_shop
npm install

# Option A: Start in Clean Mode (zero bugs planted - for baseline testing)
npm run start:clean

# Option B: Start with All 4 Planted Bugs Enabled
npm run start:all-bugs

# Option C: Start with a Single Specific Bug Enabled (PowerShell)
$env:BUGS="negative_qty"; node server.js
# BUGS=negative_qty node server.js   # Linux/macOS
```

9.3 Running the Autonomous QA Agent

From the project root (x:\QA Testing Agent) in your activated virtual environment:

```
# 1. Standard Interactive Run (Prompts user for ambiguity question)
python -m qa_agent
```

```
# 2. Non-Interactive Run (Reads pre-filled answers for CI/automation)
python -m qa_agent --answers answers.example.json

# 3. Visible Browser Run (Watches the Chromium browser window live)
python -m qa_agent --no-headless --answers answers.example.json

# 4. Custom Report Output and Call Budget
python -m qa_agent --report reports/my_custom_run.md --max-calls 40 --answers
answers.example.json
```

9.4 Running the Automated Test Suite

Run the full pytest suite covering all 22 deterministic unit and report-generation tests:

```
# Run all 22 tests with verbose output
python -m pytest tests/test_agent.py -v

# Run a specific test
python -m pytest tests/test_agent.py -k "test_loop_detector" -v
```

9.5 Locating & Interpreting Generated Artifacts

- • **reports/bug_report.md:** The human-readable Markdown summary containing the executive summary, execution table, unverified rules, and confirmed bugs.
- • **reports/run_log.jsonl:** Raw step-by-step JSONL event stream capturing observations, thoughts, actions, and results for every step.
- • **reports/test_plan.json:** Structured JSON containing the planned test cases extracted by Gemini from the specification.
- • **reports/*.png:** Visual evidence snapshots automatically linked in the bug report for confirmed bugs.

10. Hands-On Modification Tutorials

The best way to prepare for an interview is to modify the code yourself. These six progressive exercises guide you through making targeted, safe improvements grounded in the real codebase.

Exercise 1: Customize Console Log Messages & Report Metadata

- **Objective:** Understand CLI output flow and report header generation.
- **Files:** qa_agent/app.py (main) and qa_agent/agent.py (generate_markdown_report).
- **Current Behavior:** Reports display standard header: 'Autonomous QA Tester Agent — Bug Report'.
- **Desired Behavior:** Add an 'Evaluator / Tester Name' field to CLI output and the report header.

Walkthrough: Step-by-Step Instructions:

27. **1:** Open qa_agent/app.py. In main(), add a print line: print(' Evaluator: Candidate Name').
28. **2:** Open qa_agent/agent.py. In generate_markdown_report(), locate line 711. Add lines.append(f"**Tester:** {summary.get("tester", "AI Engineering Candidate")} ').
29. **3:** Run python -m pytest tests/test_agent.py -v to verify no regressions.

- **How to Undo:** Revert the lines using git checkout or undo in your editor.

Exercise 2: Add Custom Error Type Detection for DNS/Connection Failures

- **Objective:** Enhance exception classification for network drops in the execution loop.
- **Files:** qa_agent/app.py (lines 286-298).
- **Current Behavior:** Network drops default to err_type = type(e).__name__ (e.g. 'ConnectionRefusedError').
- **Desired Behavior:** Explicitly categorize connection refused errors as 'Target Server Unreachable'.

Code: Code Modification:

```
# In qa_agent/app.py around line 296, add:
elif "connection refused" in err_str.lower() or "connect econrefused" in err_str.lower():
    err_type = "Target Server Unreachable"
```

- **Verification:** Run pytest tests/test_agent.py. All 22 tests continue passing.

Exercise 3: Add a New Focused Unit Test in tests/test_agent.py

- **Objective:** Learn how to write deterministic unit tests for agent logic.
- **Files:** tests/test_agent.py.
- **Goal:** Test that an unrecognized status in _format_status_label is safely title-cased.

Code: Code to Append to tests/test_agent.py:

```
def test_format_status_label_fallback():
    from qa_agent.agent import _format_status_label
    assert _format_status_label("custom_error_state") == "Custom Error State"
```

- **Verification:** Run python -m pytest tests/test_agent.py -k 'test_format_status_label_fallback' -v. Test passes (23 total).

Exercise 4: Add a Free Shipping Rule to specs/shop.md & Server Check

- **Objective:** Extend the specification and teach app_checks() to verify a new rule.
- **Files:** specs/shop.md and qa_agent/agent.py (app_checks).
- **Step 1 (Spec):** Add: '### Rule 6: Free Shipping\nOrders with subtotal >= 500 receive free shipping; others pay 50.'
- **Step 2 (Code):** In app_checks(), check if state.get('total') correctly applies the shipping fee. If total does not match, return a BugFinding for R6.
- **Verification:** Run agent against demo app with bugs enabled; observe agent extract R6 and generate a test.

Exercise 5: Add a 'Stage' Column to the Test Execution Details Table

- **Objective:** Enhance report transparency by showing where each test stopped (e.g. setup, loop, judgment).
- **Files:** qa_agent/agent.py (generate_markdown_report).
- **Change:** Table header: | Test ID | Rule ID | Test Objective | Stage | Status | Observed Outcome / Error |
- **Implementation:** In the loop: stage = _sanitize_table_cell(r.execution_stage or 'completed'). Include stage in the row formatting.

Exercise 6: Add Exponential Backoff Retry to LLM API Calls

- **Objective:** Improve reliability during transient 503 spikes.
- **Files:** qa_agent/agent.py (LLMClient._call_llm).
- **Implementation:** Wrap client.models.generate_content in a 3-attempt loop with time.sleep(2 ** attempt) on 503 errors.

11. Comprehensive AI Engineering Interview Preparation

This section is designed to help you ace an AI Engineering Intern interview. It provides structured project pitches, 25 in-depth technical questions with model answers, and guidance on transparent AI tool disclosure.

11.1 The 30-Second Elevator Pitch

> **[30-SECOND PITCH]**
> "I built an autonomous QA tester agent that tests web applications by comparing behavior against a plain-English specification. Using Gemini for dynamic reasoning and Playwright for browser control, the agent autonomously extracts rules, identifies ambiguities, and navigates pages based on live DOM snapshots. To prevent false alarms, every suspected bug is verified through a deterministic 2/2 replay gate that resets server state and replays stable CSS locators without the LLM, culminating in a complete Markdown report with screenshot evidence."

11.2 The 2-Minute Technical Summary

> **[2-MINUTE SUMMARY]**
> "My goal was to solve the fragility and authoring cost of conventional test scripts while avoiding LLM hallucinations. The architecture is a clean 6-stage pipeline: First, Gemini parses the specification into structured rules and flags missing requirements. Second, it synthesizes bounded test cases. Third, in a 10-action agent loop, an injected JavaScript snippet extracts interactive elements into a compact snapshot, and Gemini selects actions validated by Pydantic. Playwright executes them and records stable CSS locators. Fourth, for bug judgment, I explicitly bypass the LLM—Python queries server memory directly via GET /__state to evaluate mathematical ground truth. Fifth, suspected bugs pass through an independent reproduction gate: the server resets via POST /__reset, and the recorded action sequence is replayed twice in fresh browser contexts with zero LLM involvement. Only bugs reproducing in 2/2 attempts are confirmed. Finally, the agent generates a universal Markdown report documenting all attempted tests—including passes, confirmed bugs, and upstream API interruptions."

11.3 25 Critical Interview Questions & Model Answers

Q1: Why did you build an autonomous agent instead of writing Playwright test scripts?

Conventional Playwright scripts require engineers to anticipate every path and hardcode CSS selectors, which break whenever UI styling or layouts change. Our agent reads natural-language specs, dynamically inspects live DOM snapshots, and autonomously decides actions. It adapts to element ordering and navigates alternative routes to a testing goal.

Q2: How do you prevent LLMs from hallucinating that a bug exists?

We completely exclude the LLM from bug judgment. When test actions finish, Python queries the application's internal memory via GET /__state and runs deterministic mathematical and regex checks (app_checks()). Ground truth is established by deterministic code, not probabilistic LLM judgment.

Q3: What is your 2/2 independent reproduction gate?

A suspected violation could be a transient race condition or network glitch. To ensure zero false positives, reproduce_bug() calls POST /__reset, opens a fresh browser context, and replays the recorded CSS action sequence without calling the LLM. If the bug does not reproduce in 2 out of 2 attempts, it is downgraded to 'not_reproduced' (0/2) or 'inconclusive' (1/2).

Q4: How do stable locators work, and why not use snapshot indices?

Snapshot numbers like '[3]' change if elements are reordered or dynamically loaded. When building the snapshot, our JavaScript calculates a stable CSS locator in priority order (#id > [data-testid] > [name] > text). The LLM refers to '[3]', but the agent records '#coupon-input' in RecordedAction. Replays dispatch against stable locators, ensuring rock-solid repeatability across reloads.

Q5: How do you prevent the agent from getting stuck in an infinite loop?

We enforce two safety mechanisms: First, a hard Action Budget of 10 actions per test case. Second, a deterministic Loop Detector: we hash the combined (page_state, action) string. If the agent executes the identical action on the same page state 3 consecutive times, it immediately breaks the loop.

Q6: How do you handle external LLM API failures like 503 UNAVAILABLE or 429 quota exhaustion?

In app.py, the test execution loop catches exceptions, extracts error_type and error_message, records a TestResult with status='unable_to_complete', and continues running remaining tests. The report generator documents incomplete tests under 'Unable to Complete' and notes under 'Rules Not Fully Verified' that the rule could not be conclusively evaluated.

Q7: Why do you store state in server memory instead of a database like MongoDB?

For fast, deterministic reproduction. An in-memory Express store allows POST /__reset to clear state in <5ms. Testing against MongoDB or PostgreSQL would require database migrations, transaction rollbacks, or teardown scripts, adding 5 to 10 seconds of latency to every replay attempt.

Q8: How does the agent discover requirements ambiguities?

In understand_spec(), the system prompt instructs Gemini to identify missing or vague requirements. In specs/shop.md, the maximum allowed quantity per item is deliberately omitted. The LLM notices this and sets ambiguity_question, allowing the user to provide clarification via CLI or answers.json before testing.

Q9: What happens if the LLM returns invalid JSON?

LLMClient implements a corrective retry mechanism: it catches Pydantic's ValidationError, feeds the exact error message back to Gemini in a correction prompt ('[CORRECTION REQUIRED]: Your output failed validation...'), and re-parses the corrected response. This recovers in >95% of syntax error cases without draining call budgets.

Q10: Why did you choose Playwright over Selenium or Puppeteer?

Playwright offers superior modern web primitives: built-in auto-waiting for network idle and DOM readiness, isolated browser contexts that spin up in milliseconds, and native CDP integration without requiring standalone driver binaries like chromedriver.exe.

Q11: Why didn't you use LangChain, CrewAI, or AutoGen?

Frameworks like LangChain add unnecessary abstractions that make debugging and latency optimization difficult. Writing a clean, native Python agent loop using the official Google GenAI SDK and Pydantic makes the control flow 100% transparent, explainable, and customizable.

Q12: What is the biggest limitation of your current architecture?

The deterministic assertion layer (app_checks()) is coupled to the demo store's schema. While planning, DOM snapshotting, and browser replay are completely app-agnostic, validating ground truth currently requires a custom Python check adapter. Generalizing this layer via OpenAPI/Swagger synthesis is our next step.

Q13: Does your agent perform visual regression testing?

No. It performs functional and data validation. Screenshots are captured as point-in-time visual evidence for human review, but the agent does not perform pixel-by-pixel diffing against golden baselines.

Q14: How is your system sandboxed against security risks?

The browser controller strictly validates navigation targets using is_localhost_url(). Any URL pointing outside localhost (e.g. external domains, phishing targets, or third-party OAuth links) is rejected immediately, preventing data exfiltration.

Q15: How does Pydantic v2 improve agent reliability?

Pydantic enforces strict runtime validation. In AgentAction, action is typed as a Literal. If the LLM generates a non-existent action like 'scroll_down' or 'hover', Pydantic rejects it immediately, preventing runtime crashes inside Playwright.

Q16: How many unit tests do you have and what do they verify?

We have 22 unit and regression tests in tests/test_agent.py. They verify Pydantic rejection of invalid actions, localhost sandboxing, call budget enforcement, JSON fence stripping, corrective retries, answers.json resolution, loop detection, cart total math checks, inconclusive replay gates, report generation structure, Node server resets, stable locator replays, and all 8 universal report requirements.

Q17: What was the result of your clean app baseline run?

In reports/eval_clean_run1.md, the agent ran against the demo store with zero planted bugs. All 4 tests passed, and 0 bugs were reported (0.0% false positive rate), demonstrating the effectiveness of the 2/2 replay gate.

Q18: What was the result of your all-bugs run?

In reports/eval_allbugs_run1.md, all 4 planted bugs (negative quantity, duplicate coupon, stale total, invalid email) were detected and confirmed in 2/2 replays using 25 LLM calls.

Q19: If an LLM call fails during test 2 of 5, does the agent crash?

No. The try...except block in app.py catches the exception, logs it in TestResult as 'unable_to_complete', and safely proceeds to tests 3, 4, and 5. The final report is always generated.

Q20: How would you scale this to test hundreds of pages?

By parallelizing test execution using asynchronous Playwright workers or pytest-xdist across isolated browser contexts, and caching repeated DOM snapshots to reduce LLM calls.

Q21: How do you differentiate a passed test from an incomplete test in your report?

In the Executive Summary and Test Execution Details table, passed tests are marked 'Passed' (indicating spec compliance), whereas interrupted tests are marked 'Unable to Complete' with the raw error message and listed under 'Rules Not Fully Verified'.

Q22: Why do you compress the DOM into a snapshot instead of sending screenshots to a multimodal model?

Text DOM snapshots are ~500 tokens, process in <1 second, and provide exact element IDs and attributes. Multimodal screenshots consume ~1,500 tokens, have higher latency, and require vision models to guess element coordinates, which is prone to pixel-clicking inaccuracies.

Q23: How do you configure planted bugs in the demo application?

Via the BUGS environment variable on the Node server (e.g. BUGS=coupon_twice,negative_qty,stale_total,bad_email). The server checks process.env.BUGS to toggle buggy branches in its API routes.

Q24: What is the purpose of answers.example.json?

It provides pre-filled answers to specification ambiguities, enabling non-interactive test runs in CI pipelines or automated benchmark suites without requiring human terminal input.

Q25: How did you use AI coding assistants during this project?

I used Antigravity IDE and Gemini Coding Assistant as an interactive pair programmer for architecture scaffolding, test authoring, and debugging. All design decisions, deterministic guardrails, and validation logic were deliberately structured to ensure full explainability.

11.4 Transparent AI Disclosure Guidelines for Interviews

Modern engineering teams value candidates who leverage AI tools effectively while understanding every line of code. Here is the recommended script for disclosing AI tool usage during an interview:

> **[ETHICAL & TRANSPARENT AI DISCLOSURE]**
> "I used Google Antigravity IDE with Gemini as an AI pair programming assistant to help scaffold initial boilerplate and generate regression test cases. However, the architectural design—including the hybrid autonomous/deterministic separation, the DOM snapshotting engine, the 2/2 replay confirmation gate, and the error-handling pipeline—was deliberately engineered to satisfy strict QA requirements. I understand every function, can walk through the codebase, and am prepared to debug or modify it live."

12. Self-Assessment, Technical Glossary & Mastery Checklist

12.1 Technical Glossary

- • **Autonomous Agent:** A software entity that perceives its environment through sensors/DOM, reasons using an LLM, and takes autonomous actions to achieve goals.
- • **DOM Snapshot:** A compact, filtered text representation of interactive web page elements, discarding non-interactive boilerplate to conserve LLM tokens.
- • **Stable Locator:** A resilient CSS selector (such as #id or [data-testid]) calculated from DOM attributes that remains valid across page reloads and layout changes.
- • **Deterministic Replay:** Executing a sequence of recorded browser actions using stable locators without an LLM, guaranteeing exact repeatability.
- • **2/2 Replay Gate:** A verification threshold requiring a suspected bug to reproduce in two consecutive fresh browser contexts after state reset before being marked confirmed.
- • **Ground-Truth Oracle:** An infallible verification mechanism (here, Python mathematical assertions on server memory via GET /__state) that determines whether a bug occurred.
- • **Action Budget:** A hard upper bound (10 actions) preventing an agent from wandering indefinitely during a single test case.
- • **Loop Detector:** A safety check that terminates test execution if an agent repeats the identical action on the same page state 3 times consecutively.
- • **Pydantic Schema:** A type-enforced Python class inheriting from BaseModel that validates JSON structures and enforces runtime data contracts.
- • **In-Memory State:** Application state stored in server RAM (plain JS objects) rather than a persistent database, enabling instantaneous (<5ms) state resets.
- • **Specification Ingestion:** The process where an LLM reads natural-language requirements (Markdown) and extracts structured rules with verification criteria.
- • **Ambiguity Detection:** The agentic capability to identify unstated, vague, or conflicting requirements in a spec and ask targeted clarification questions.
- • **Inconclusive Finding:** A suspected specification violation that reproduced intermittently (e.g. 1/2 replays) and is reported as unconfirmed.
- • **Unable to Complete:** A test execution status indicating that the test halted prematurely due to external infrastructure failures (e.g. API 503) rather than a bug.
- • **Rules Not Fully Verified:** A dedicated report section identifying rules whose tests did not reach conclusive outcomes due to premature execution termination.
- • **Browser Context:** An isolated browser session in Playwright with independent cookies, cache, and storage, equivalent to an incognito window.
- • **Corrective Retry:** A pattern where a schema validation error is fed back to the LLM in a secondary prompt to obtain corrected JSON.
- • **Localhost Sandbox:** A security constraint blocking navigation to any origin other than localhost, preventing external data leakage.
- • **Call Budget:** A global cap (default 60 calls) restricting the total number of LLM API requests permitted per evaluation run.
- • **Stepwise Run Log:** A chronological JSONL file recording timestamps, snapshots, thoughts, actions, and observations for every agent step.

12.2 Knowledge Check Quiz

1. Why does the agent record stable CSS locators instead of snapshot numbers [N] for replay?

Answer: Snapshot numbers are ephemeral and shift when element order changes. Stable locators (#id) remain valid across page reloads and fresh browser contexts.

2. If a test halts due to Gemini returning '503 UNAVAILABLE', what is its status in the report?

Answer: It is classified as 'Unable to Complete' and listed under 'Rules Not Fully Verified'. It is NEVER counted as a pass or a failure.

3. Why is the LLM excluded from the bug judgment stage?

Answer: To prevent hallucinated bug reports. Bug judgment is performed by deterministic Python code (app_checks()) querying ground-truth server memory via GET /__state.

4. What condition triggers the agent's Loop Detector?

Answer: Executing the exact same action on the exact same page state for 3 consecutive iterations.

5. What is the maximum number of actions the agent is permitted to execute per test case?

Answer: 10 actions (enforced by MAX_ACTIONS_PER_TEST).

6. What endpoint resets the Express demo store's state?

Answer: POST /__reset (resets cart, coupons, and registered emails in <5ms).

7. How does the agent handle specification ambiguities?

Answer: It detects missing requirements during understand_spec(), asks an ambiguity question, and resolves it via CLI prompt or answers.json.

8. How many unit tests currently exist in tests/test_agent.py?

Answer: 22 unit and regression tests.

9. What model is configured by default for Gemini?

Answer: gemini-2.5-flash (with fallbacks to gemini-3.7-flash and gemini-2.5-flash-lite).

10. What does a replay result of '1/2' indicate?

Answer: The violation was intermittent and is categorized as 'inconclusive' (not confirmed).

12.3 Mastery Checklist: Concepts to Explain Without Notes

- [ ] Explain the 6-stage agent pipeline from spec reading to report generation.
- [ ] Explain the difference between DOM snapshotting and visual screenshot testing.
- [ ] Explain how resolve_target() priority logic works (#id > data-testid > name > text).
- [ ] Explain why Pydantic validation is essential for LLM action reliability.
- [ ] Explain the corrective retry mechanism in LLMClient.
- [ ] Explain why server-state verification via GET /__state eliminates hallucinations.

- **[ ]** Explain why 2/2 replay confirmation reduces false-positive risk.
- **[ ]** Explain why incomplete tests are classified as 'Unable to Complete' instead of failed.
- **[ ]** Explain how the Loop Detector and Action Budget prevent runaway execution.
- **[ ]** Explain how the localhost security sandbox protects against external navigation.
- **[ ]** Explain how in-memory Express state enables sub-5ms deterministic resets.
- **[ ]** Explain how ambiguity questions are extracted and answered non-interactively.
- **[ ]** Explain the structure of the universal Markdown report.
- **[ ]** Explain how to diagnose and recover from a Gemini 503 UNAVAILABLE error.
- **[ ]** Explain how to run pytest and what the 22 unit tests verify.